

SCADE Model Parser

Design Document

Version 1.0 | May 2026

Document Owner: Engineering Team

Classification: Internal

Status: Released

Created: May 2026

Last Revised: May 2026

1 Introduction

1.1 Purpose

This Design Document describes the functional requirements, data model, module structure, error-handling strategy, and key design decisions behind the SCADE Model Parser. It is intended for software engineers who maintain, extend, or integrate the parser.

1.2 Scope

The SCADE Model Parser is a command-line Python utility that reads one or more SCADE Suite model files (.scade / .xscade) and extracts three categories of information:

- Input signals — variables declared in the input interface of an operator.
- Output signals — variables declared in the output interface of an operator.
- Calibration parameters — Sensor and Const/Constant elements that represent tuneable values or physical constants.

Extracted data is written to a formatted multi-sheet Microsoft Excel workbook (.xlsx). The parser is designed for integration into a model-based development (MBD) workflow where SCADE Suite is used for safety-critical embedded software.

1.3 Definitions

Term	Definition
SCADE	Safety-Critical Application Development Environment — a model-based design tool by Ansys.
.scade	XML-based file format used by SCADE Suite to store operator and package definitions.
Operator	A SCADE functional unit (node or function) with a defined input/output interface.
Sensor	A SCADE element representing a calibration parameter that may be set by an external calibration tool.
Const	A SCADE element representing a compile-time constant used as a calibration parameter.
XXE	XML External Entity injection — a class of XML parser attack mitigated by this parser.
DXA	Document eXtension Attribute — the unit system used by Word documents (1440 DXA = 1 inch).

2 Requirements

2.1 Functional Requirements

ID	Category	Requirement	Priority
FR-01	Input	Accept a single .scade or .xscade file as input.	Must
FR-02	Input	Accept a directory path and recursively parse all .scade / .xscade files within it.	Must
FR-03	Parsing	Extract all input signals (Name, Type, Operator, Package, Comment) from each parsed file.	Must
FR-04	Parsing	Extract all output signals with the same attributes as input signals.	Must
FR-05	Parsing	Extract all Sensor and Const/Constant elements as calibration parameters, including Default Value.	Must
FR-06	Parsing	Handle all three SCADE XML input/output declaration patterns: flat attribute, Variable child element, and Direction attribute.	Must
FR-07	Parsing	Strip XML namespace prefixes so parsing is version-agnostic.	Must
FR-08	Output	Write results to a multi-sheet .xlsx workbook: Summary, Input Signals, Output Signals, Calibration Parameters.	Must
FR-09	Output	Include source file name in every extracted record.	Must
FR-10	Output	Support a user-specified output file path via CLI flag.	Should
FR-11	Output	Apply alternating row colours, frozen header rows, and auto-filter to all data	Should

		sheets.	
FR-12	CLI	Provide a --help option describing all arguments.	Must
FR-13	Reporting	Print a summary of record counts to stdout after parsing.	Should
FR-14	Safety	Reject any input file that exceeds 50 MB before attempting to parse it; report the skip to stderr and continue with remaining files.	Must
FR-15	Safety	Only open files whose extension is .scade or .xscade; silently skip (with a stderr message) any file with a different extension found during directory scanning.	Must

2.2 Non-Functional Requirements

ID	Category	Requirement	Priority
NFR-01	Security	Block XML External Entity (XXE) injection. Use defusedxml if available; otherwise reject DOCTYPE declarations.	Must
NFR-02	Security	Reject files exceeding 50 MB to guard against decompression/parsing attacks.	Must
NFR-03	Security	Validate all file extensions against an allowlist before opening.	Must
NFR-04	Security	Sanitise the output filename; reject any path whose directory component does not exist.	Must
NFR-05	Security	Never use eval(), exec(), or shell=True in any code path.	Must
NFR-06	Performance	Parse a 1 MB .scade file in under 2 seconds on standard hardware.	Should
NFR-07	Reliability	Skip individual files that fail to parse and continue processing	Must

		remaining files; report errors to stderr.	
NFR-08	Compatibility	Support Python 3.10 and above.	Must
NFR-09	Compatibility	Produce .xlsx files compatible with Microsoft Excel 2016+ and LibreOffice Calc 7+.	Must
NFR-10	Maintainability	Keep all code in a single self-contained Python file with no non-stdlib dependencies beyond openpyxl and defusedxml.	Should
NFR-11	Extensibility	Allow new SCADE element types to be supported by adding a single elif branch in the <code>_walk()</code> method.	Should

3 Data Model

Internally, all extracted records are represented as plain Python dictionaries. There are two record schemas — one shared by input and output signals, and one for calibration parameters.

3.1 Signal Record

Field	Type	Description
Name	str	Signal variable name as declared in the SCADE model.
Type	str	SCADE type reference (e.g. float32, bool, int32). 'unknown' if not found.
Operator	str	Name of the enclosing SCADE operator (empty if declared at package level).
Package	str	Fully-qualified package path (e.g. VehicleControl/BrakeSubsystem).
Comment	str	Optional human-readable description from the model (may be empty).
Source File	str	Filename of the .scade file from which this record was extracted.

3.2 Calibration Parameter Record

Field	Type	Description
Name	str	Calibration parameter name as declared in the SCADE model.
Type	str	SCADE type reference (e.g. float32, int32).
Kind	str	'Sensor' or 'Constant' — indicates the SCADE element type.
Default Value	str	Initial or default value if present in the model (may be empty).
Operator	str	Name of the enclosing operator (often empty for top-level sensors/constants).
Package	str	Fully-qualified package path.
Comment	str	Optional description from the model.
Source File	str	Source .scade filename.

4 Data Flow

The following steps describe the end-to-end data flow from user invocation to Excel output.

- 1. CLI Invocation** The user runs `scade_parser.py` with an input path and optional `--output` flag. `argparse` parses the arguments; `_build_arg_parser()` defines the interface.
- 2. Path Resolution & Validation** The input path is resolved to an absolute path. If it is a file, it is placed in a single-element list. If it is a directory, `rglob()` discovers all `.scade` and `.xscade` files. Each file is validated for extension and size before opening.
- 3. XML Parsing** `ScadeModelParser.__init__()` stores the file path. `.parse()` calls `defusedxml` (or the `stdlib` fallback) to build an `ElementTree`. The root element is passed to `_walk()`.
- 4. Tree Walk** `_walk()` performs a depth-first traversal of the XML element tree. At each element, the tag name is stripped of its namespace prefix and matched against known SCADE element names. Context variables (current operator name, current package path) are threaded through the recursion.
- 5. Record Collection** `_collect_var()` and `_collect_cal()` build dictionary records from element attributes and child text nodes. Records are appended to `self.inputs`, `self.outputs`, or `self.calibrations` as appropriate.
- 6. Aggregation** `parse_path()` iterates over all discovered files. Results from each `ScadeModelParser` instance are concatenated into three master lists.
- 7. Excel Generation** `write_excel()` creates an `openpyxl` `Workbook`, adds four sheets, applies formatting, freezes headers, and saves to the specified output path.
- 8. Summary Output** Record counts per category are printed to `stdout`. Any per-file errors encountered during step 3–5 are reported to `stderr`.

5 Module and Class Design

5.1 Module Overview

scade_parser.py is structured as a single self-contained module with the following logical sections, in order of appearance:

- Imports and dependency management (defusedxml / stdlib fallback, openpyxl guard)
- Constants (MAX_FILE_SIZE_MB, ALLOWED_EXTENSIONS, Excel style objects)
- Path and input validation helpers (_validate_input_path, _validate_output_path, _sanitise_filename)
- XML utility functions (_strip_ns, _attr, _child_text)
- ScadeModelParser class — core parsing logic
- parse_path() — multi-file aggregation
- Excel writer (write_excel, _write_data_sheet, _write_summary_sheet)
- CLI entry point (main, _build_arg_parser)

5.2 ScadeModelParser

The central class. Instantiated once per file. Responsible for all XML traversal and record collection.

Method	Visibility	Description
__init__(file_path)	Public	Stores the resolved file path; initialises empty inputs, outputs, calibrations lists.
parse()	Public	Validates the path, calls _parse_xml(), passes the root to _walk(). Returns self for method chaining.
_walk(element, operator, package)	Private	Depth-first recursive tree walk. Updates context variables and dispatches to _collect_var() or _collect_cal().
_collect_var(el, direction, ...)	Private	Builds a signal dict from an Input or Output element and appends it to the correct list.
_collect_cal(el, kind, ...)	Private	Builds a calibration dict from a Sensor or Const element. Resolves default values from attributes and child text.

5.3 parse_path()

A module-level function (not a class method) that accepts a Path and returns three aggregated lists. It handles both single-file and directory cases, instantiates a ScadeModelParser for each file, and catches per-file exceptions so that one bad file does not abort the entire batch.

5.4 Excel Writer Functions

Function	Responsibility
write_excel(inputs, outputs, calibrations, path)	Orchestrates sheet creation; calls

	_write_data_sheet for each data sheet then _write_summary_sheet.
_write_data_sheet(ws, columns, rows)	Writes a formatted data sheet: header row (navy background, white text), alternating row fill, auto-width columns, freeze pane, auto-filter.
_write_summary_sheet(wb, ...)	Inserts a Summary sheet at index 0 with generation timestamp, output filename, and record counts per category.

6 Error Handling Strategy

The parser applies a two-tier error handling strategy to balance strictness (catching configuration mistakes early) with robustness (graceful degradation over batches of files).

6.1 Fatal Errors — Abort Immediately

The following conditions terminate the process with a non-zero exit code:

- Invalid or non-existent output directory.
- Output filename with a non-.xlsx extension.
- Input path that does not exist (single-file mode).
- No write permission for the output directory.

6.2 Per-File Errors — Skip and Continue

When processing a directory, errors on individual files are caught, reported to stderr with a [SKIP] prefix, and processing continues:

- XML parse errors (malformed XML, invalid encoding).
- Files exceeding the 50 MB size limit.
- Files with an extension not in the allowlist.
- Files with restricted read permissions.

6.3 Warnings

If parsing completes but no records are extracted (e.g. the XML uses non-standard element names), a [WARNING] message is printed to stderr and the process exits with code 0. No Excel file is written in this case.

7 Key Design Decisions

Single-file module

All code lives in `scade_parser.py` to simplify deployment — no package structure, no `__init__.py`, no relative imports. This is appropriate for a utility script that will be shared and run directly. The trade-off is reduced modularity, which is acceptable given the bounded scope.

defusedxml with stdlib fallback

`defusedxml` is the recommended choice for XXE protection. However, requiring it as a hard dependency would prevent use in locked-down environments. The fallback disables the most common attack vector (external entity resolution) using `stdlib` mechanisms; it also rejects `DOCTYPE` declarations to block billion-laughs attacks. A warning is printed when `defusedxml` is absent.

Plain dict records (not dataclasses)

Records are Python dicts rather than dataclasses or named tuples. This keeps the code simpler, avoids class-per-schema boilerplate, and makes it straightforward to add new fields without changing a class definition. The downside is lack of compile-time type safety, which is acceptable for a parsing utility.

Recursive tree walk with context threading

The `_walk()` method threads operator and package context variables through recursive calls rather than maintaining a stack. This keeps the code straightforward for moderately nested SCAD models. Deeply nested Package hierarchies are supported because the package path is built incrementally as the walk descends.

openpyxl for Excel output

`openpyxl` is the de-facto standard for `.xlsx` generation in Python, is actively maintained, and produces files compatible with Excel 2016+. Alternative libraries (`xlswriter`, `xlwt`) were considered; `xlswriter` produces slightly smaller files but does not support reading existing workbooks — a potential future requirement. `openpyxl` was chosen for versatility.

Multi-attribute fallback in `_attr()`

SCADE XML attribute naming is not perfectly consistent across model versions (e.g. `TypeRef` vs `typeRef` vs `Type`). The `_attr()` helper accepts a list of candidate attribute names and returns the first match. This makes the parser resilient to minor schema variations without requiring version detection logic.

Column width cap at 55 characters

Auto-sizing columns to their content maximum could produce very wide columns for long comment strings. A cap of 55 characters keeps the workbook readable without requiring manual adjustment in Excel.

8 Supported SCADE XML Patterns

The parser recognises three syntactic patterns for declaring input and output signals in SCADE XML, as well as two patterns for calibration parameters.

8.1 Pattern A — Flat Attribute (Inline Signal)

Used by SCADE Suite when signals are declared directly on the <Input> or <Output> element:

```
<Input Name="BrakePedalPos" TypeRef="float32" Comment="Pedal position 0-100%"/>
<Output Name="BrakePressure" TypeRef="float32"/>
```

8.2 Pattern B — Variable Children

Used when the interface block groups signals under <Input> or <Output> parent elements:

```
<Input>
  <Variable Name="AccelPedalPos" TypeRef="float32"/>
  <Variable Name="EngineSpeed_rpm" TypeRef="int32"/>
</Input>
```

8.3 Pattern C — Direction Attribute

Used in some SCADE versions where direction is an attribute of the <Variable> element:

```
<Variable Name="ThrottleCmd" TypeRef="float32" Direction="out"/>
```

8.4 Calibration: Sensor

Sensor elements represent calibration parameters adjustable at run time:

```
<Sensor Name="CAL_MaxSpeed" TypeRef="float32">
  <Default>120.0</Default>
</Sensor>
```

8.5 Calibration: Const / Constant

Constant elements represent compile-time-fixed calibration values:

```
<Const Name="K_GAIN" TypeRef="float32" Value="2.5"/>
<Constant Name="K_PI" TypeRef="float64">
  <Value>3.14159265</Value>
</Constant>
```

9 Revision History

Version	Date	Author	Summary of Changes
1.0	May 2026	Engineering Team	Initial release.